

PYTHON FOR DATA SCIENCE • MODULE 1 • TOPIC 1.8

Inheritance & Polymorphism

Day 8 — building on a class you already wrote

90 min

Concepts + live code

30 min

Practice notebook

1

New function: `super()`



Boston University

Prepared & presented by **Srinivasa Sai Chava**

Before we start — five questions from Day 7

- 1 What is `self`, in one sentence? → The object itself, passed in as the first parameter.
- 2 What is `ravi.speak()` really? → `Student.speak(ravi)` — Python fills the first slot.
- 3 Why is a list in the class body a bug? → One list, shared by every object. Put it in `__init__`.
- 4 What does `d.speak` (no brackets) give? → The method object. You never called it.
- 5 What does `__str__` control? → What `print(obj)` displays.

Everything today rests on self. If question 1 felt shaky, say so now — the next 90 minutes will be hard otherwise.

How today's two hours are structured

1

Inheritance

Reusing a class you already wrote

25 min

2

Overriding & super()

Replacing or extending a parent method

25 min

3

Polymorphism

One call, many behaviours

20 min

4

Multiple inheritance

Two parents, the MRO, and when to stop

15 min

5

Mini build

An Employee hierarchy

5 min

6

Practice notebook

You code, I walk around

30 min

The single most important line of the day is `super().__init__(...)` in section 2.

By the end of today you will be able to

1

Inherit

Build a child class that reuses everything a parent already has

2

Override

Replace a parent's method, or extend it with `super()`

3

Generalise

Write one loop that works on several different classes

4

Restrain

Know when inheritance is the wrong tool, and use composition instead

Where you will see this in three weeks

scikit-learn has hundreds of models, and every one of them answers `.fit()` and `.predict()`. That is exactly what you learn today: many classes, one shared set of method names, and code that works with any of them without knowing which it received.

1

Inheritance

Reusing a class you already wrote · 25 minutes

- The duplication problem
- `class Child(Parent)`
- What gets inherited
- `isinstance`

The problem inheritance solves

Two classes, mostly identical

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def greet(self):
        return f"Hi, I am {self.name}"

class Teacher:
    def __init__(self, name, age):    # identical
        self.name = name            # identical
        self.age = age              # identical
    def greet(self):                # identical
        return f"Hi, I am {self.name}"
```

One parent, two children

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def greet(self):
        return f"Hi, I am {self.name}"

class Student(Person):    # gets everything
    pass

class Teacher(Person):    # gets everything
    pass
```

Day 3's lesson, one level up. Functions stopped you repeating a calculation. Inheritance stops you repeating a whole class. Fix `greet()` once in `Person` and every child is fixed — which is the same argument, applied to a bigger unit of code.

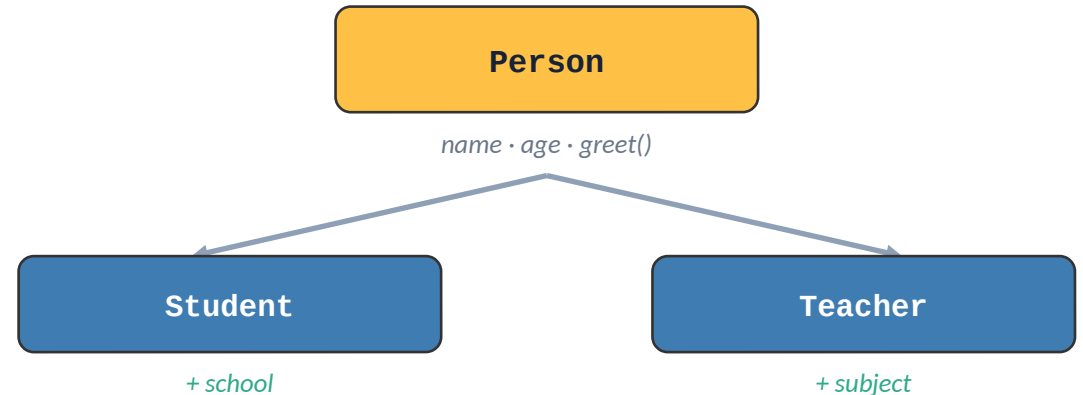
class Child(Parent) — and what comes with it

```
class Person:  
    ...  
  
class Student(Person):  
    school = "BU"
```

The child gets, for free:

- every method the parent defines
- every attribute `__init__` sets
- every class attribute

The family tree



Arrows point from child to parent: "Student IS A Person".

Checking the family: `isinstance` and `issubclass`

```
isinstance(ravi, Student) -> True  
isinstance(ravi, Person) -> True    also!  
issubclass(Student, Person) -> True
```

A Student really is a Person — not merely similar to one. That is why any code written to accept a Person will happily accept a Student, which is the whole basis of section 3.

2

Overriding & super()

Replacing or extending · 25 minutes

- Method overriding
- `super()`
- `super().__init__`
- The lookup order

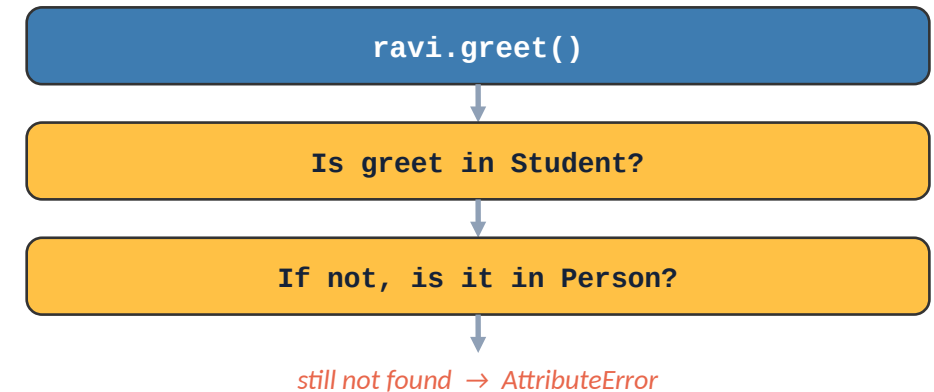
Overriding — the child's version wins

```
class Person:
    def greet(self):
        return "Hi, I am a person"

class Student(Person):
    def greet(self):           # same name
        return "Hi, I am a student" # replaces it

Person().greet()    -> "...a person"
Student().greet()   -> "...a student"
```

How Python finds a method



Python stops at the FIRST match — so the child always wins.

Day 3's LEGB rule has a cousin

Names are found by walking a chain and stopping at the first hit. For variables that chain is Local → Enclosing → Global → Built-in. For attributes it is the object → its class → its parent → upwards to object itself. Same idea, different chain — and it is why a child's method shadows the parent's exactly as a local variable shadows a global.

super() — extending instead of replacing

Overriding throws the parent's work away

```
class Person:
    def greet(self):
        return f"Hi, I am {self.name}"

class Student(Person):
    def greet(self):
        return "Hi, I study at BU"
        # the name is gone from the message
```

super() runs the parent's version first

```
class Student(Person):
    def greet(self):
        base = super().greet()
        return base + ", I study at BU"

# "Hi, I am Ravi, I study at BU"
```

⚠ The line you must not forget: super().__init__(...)

```
class Student(Person):
    def __init__(self, name, age, school):
        super().__init__(name, age) # parent's setup
        self.school = school        # then your own
```

Writing your own `__init__` REPLACES the parent's completely. Without that first line, `name` and `age` are never set, and the first thing that reads `self.name` dies with an `AttributeError` — often far away from the real cause. Call `super().__init__` first, then add what is new.

3

Polymorphism

One call, many behaviours · 20 minutes

- The same method name
- One loop, many types
- Duck typing

Polymorphism — one call, many behaviours

```
class Animal:
    def speak(self):
        return "..."

class Dog(Animal):
    def speak(self): return "Woof"

class Cat(Animal):
    def speak(self): return "Meow"

for a in [Dog(), Cat(), Dog()]:
    print(a.speak())          # Woof / Meow / Woof
```

Why this matters

The loop never asks what kind of animal it has. It just calls `speak()` and each object answers in its own way.

```
# WITHOUT polymorphism:
if isinstance(a, Dog):
    print("Woof")
elif isinstance(a, Cat):
    print("Meow")
# ...and one more branch per new animal
```

The real benefit is what happens when you add a new class

Adding a Cow means writing the Cow class and nothing else — the loop already works. In the if/elif version you must find and edit every branching block in the codebase, and any one you miss becomes a bug. Polymorphism moves the decision from your code into the objects themselves.

Duck typing — Python does not even require inheritance

No shared parent, and it still works

```
class Dog:                # no parent
    def speak(self): return "Woof"

class Robot:              # no parent
    def speak(self): return "Beep"

for x in [Dog(), Robot()]:
    print(x.speak())      # Woof / Beep
```

“If it walks like a duck...”

Python never checks an object's type before calling a method. It simply tries. If the object has a `speak()` method, the call works — the family tree is irrelevant.

This is why `len()` works on a string, a list, a dict and your own class — anything that defines `__len__`.

So when is inheritance still worth it?

Shared code

the parent holds the logic all children need

A shared type

`isinstance(x, Animal)` becomes meaningful

A contract

the parent documents what every child must provide

Duck typing gives you polymorphism without inheritance. Inheritance gives you shared implementation on top. Use inheritance when the children genuinely share code — not merely because they share a method name.

4

Multiple Inheritance

Two parents, and knowing when to stop · 15 minutes

- Two parents
- The MRO
- Composition instead

Two parents, and the order Python searches

```
class Swimmer:
    def move(self): return "swims"

class Flyer:
    def move(self): return "flies"

class Duck(Swimmer, Flyer):    # two parents
    pass

Duck().move()    -> "swims"
#                Swimmer came FIRST
```

The Method Resolution Order

Python builds one flat search order from the class and its parents, left to right, and stops at the first match. You can always ask to see it:

```
Duck.__mro__

(Duck, Swimmer, Flyer, object)
# ^ searched in this order
```

Multiple inheritance gets confusing quickly

With two parents the order is easy to read. With four parents that themselves have parents, working out which `move()` runs becomes a research exercise — and a reader of your code has to do it too. Most professional Python uses single inheritance almost exclusively. Recognise multiple inheritance when you meet it in a library; think hard before writing it yourself.

When inheritance is the wrong tool

⚠ Inheriting just to reuse code

```
class Engine:
    def start(self): return "vroom"

class Car(Engine):          # a Car IS an Engine?
    pass
```

It works — `car.start()` runs — but the claim is false. A car is not a kind of engine, and every later reader has to absorb that lie.

Composition: the object HAS one

```
class Car:
    def __init__(self):
        self.engine = Engine()    # HAS an engine

    def start(self):
        return self.engine.start()
```

Now the relationship is honest, and you can swap in an `ElectricEngine` without touching the family tree.

The test, before you write `class Child(Parent):`

Say it out loud: “a Student IS A Person” — true, so inherit. “a Car IS AN Engine” — false, so give the car an engine instead. If the sentence sounds wrong to an ordinary English speaker, the inheritance is wrong too, however convenient the code reuse looks.

Debug corner — eight inheritance bugs

You wrote	What happens	Fix
<code>child __init__, no super()</code>	parent attributes never set	<code>super().__init__(...)</code>
<code>super().__init__() last</code>	your attributes may be overwritten	call it FIRST
<code>Person.__init__(name)</code>	TypeError, self missing	<code>super().__init__(name)</code>
<code>class Student(Person)</code>	forgot the brackets entirely	brackets name the parent
<code>def greet():</code> in a child	TypeError on the call	<code>def greet(self):</code>
<code>super.greet()</code>	no brackets on super	<code>super().greet()</code>
<code>class Car(Engine)</code>	works, but the claim is false	give the Car an engine
<code>obj.method()</code>	AttributeError up the chain	check the parent actually defines it

Rows 1 to 3 are all the same root cause: a child `__init__` that does not correctly hand off to its parent. All eight are in today's practice notebook.

Putting it together — an Employee hierarchy

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary
    def pay(self):
        return self.salary / 12
    def __str__(self):
        return f"{self.name}: {self.pay():.0f}/month"

class Manager(Employee):
    def __init__(self, name, salary, bonus):
        super().__init__(name, salary)
        self.bonus = bonus
    def pay(self):
        return super().pay() + self.bonus

class Intern(Employee):
    def pay(self):
        return 1000

staff = [Employee("Ravi", 60000),
         Manager("Sara", 90000, 500),
         Intern("Amit", 0)]

for person in staff:
    print(person)
```

Parent holds shared code
name, salary, __str__

super().__init__ first
then self.bonus

Extend with super()
Manager.pay adds a bonus

Replace entirely
Intern.pay ignores salary

Polymorphism
one loop, three behaviours

__str__ inherited
all three print the same way

__str__ calls pay()
and gets the CHILD's version

The last note is the deepest: `Employee.__str__` calls `self.pay()`, and `self` is a `Manager` — so the child's `pay()` runs.

Practice session — 30 minutes



Predict the output

Overriding, super(), MRO, isinstance

8 questions



Spot & fix the bug

Missing super(), wrong order, no self

4 questions



Write the code

Shapes, vehicles, accounts, animals

5 tasks

Your notebook

[Day8_Practice_Questions.ipynb](#)

- Write the parent first
- Then one child at a time
- Raise a hand — I am walking around
- Solutions shared after class

If a child object is missing an attribute, you forgot `super().__init__()`. That one line explains most of what will go wrong in the next half hour.

TODAY IN TWELVE LINES

Recap

- 1 class Child(Parent) inherits every method and attribute.
- 2 A child method with the same name overrides the parent's.
- 3 Python searches the child first, then up the chain.
- 4 super() calls the parent's version instead of replacing it.
- 5 A child __init__ MUST call super().__init__(...) first.
- 6 isinstance(child_obj, Parent) is True — it really is one.
- 7 Polymorphism: one loop, many classes, no type checks.
- 8 Adding a new class needs no change to existing loops.
- 9 Duck typing: a shared method name is enough, no parent needed.
- 10 Two parents are searched left to right — see __mro__.
- 11 A parent calling self.method() gets the CHILD's version.
- 12 “IS A” → inherit. “HAS A” → composition.

Homework

Build a Shape parent with Circle and Square children, each with area() · Extend Day 7's BankAccount into a SavingsAccount that adds interest · Find one place inheritance would be wrong

Next class — Topic 1.9

File I/O: reading and writing text and CSV files, the with statement, and file modes